FIAP ON

IA COMO FERTILIZANTE DIGITAL - UM NOVO AGRONEGÓCIO DO AMANHÃ

INTELIGÊNCIA ARTIFICIAL COMO SERVIÇO: EXPLORANDO OS RECURSOS COGNITIVOS DA AWS

7

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Solução multilíngue 6
Figura 2 - Árvore Sintática 7
Figura 3 - Evolução de modelos de NLP 8
Figura 4 - Transcrição em tempo real 11
Figura 5 - Buckets de entrada e saída 12
Figura 6 - Criação de tarefa de transcrição 12
Figura 7 - Definição de tarefa de transcrição 13
Figura 8 - Definição de idioma de entrada 13
Figura 9 - Definição de bucket de entrada e saída 14
Figura 10 - Tarefa de transcrição 14
Figura 11 - Tarefa concluída 15
Figura 12 - Conteúdo da transcrição 15
Figura 13 - Conteúdo do arquivo de legendas 16
Figura 14 - Tradução por meio do Amazon Translate 17
Figura 15 - Tradução em tempo real 17
Figura 16 - Tradução de um documento 18
Figura 17 - Topic Modelling 19
Figura 18 - Identificação de entidades 19
Figura 19 - Motores do Amazon Polly 21
Figura 20 - Amazon Bedrock 21
Figura 21 - Seleção de modelo fundacional 22
Figura 22 - Playground de Texto 22
Figura 23 - Mudança de modelo de LLM 23
Figura 24 - Instâncias de Notebooks 24
Figura 25 - Tela do Jupyter 24
Figura 26 - Notebook em Python 25
Figura 27 - Criação de bloco de anotações 25
Figura 28 - Acesso ao JupyterLab 26
Figura 29 - Ambiente do JupyterLab 26
Figura 30 - Recursos do JupyterLab 27
Figura 31 - Exemplo de Notebook no JupyterLab 28
Figura 32 - DataFrame de exemplo 29
Figura 33 - Sentimento identificado pelo Comprehend 29
Figura 34 - Solução de NLP para classificar sentimentos 30
Figura 35 - Notebook para análise de sentimentos criado 31
Figura 36 - Conteúdo do dataset 32
Figura 37 - Dataset após limpeza 35

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Importação de bibliotecas 31
Código-fonte 2 - Visualização dos dados com Pandas 32
Código-fonte 3 - Frequência dos sentimentos nos dados em análise 33
Código-fonte 4 - Limpeza de ruídos 33
Código-fonte 5 - Classe para limpeza de dados 34
Código-fonte 6 - Limpeza de ruídos 35
Código-fonte 7 - Treinamento do modelo 36
Código-fonte 8 - Predição em novos dados 36
Código-fonte 9 - Código completo da solução de NLP 39

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 INTELIGÊNCIA ARTIFICIAL COMO SERVIÇO: EXPLORANDO OS RECURSOS COGNITIVOS DA AWS 5
1.1 Projeto 6
2 PROCESSAMENTO DE LINGUAGEM NATURAL 7
2.1 Análise sintática 7
3 SOLUÇÕES DE NLP NA AWS 10
3.1 Amazon Transcribe 10
3.2 Uso do Transcribe 11
3.3 Amazon Translate 16
3.4 Amazon Comprehend 18
3.5 Amazon Polly 20
3.6 Amazon Bedrock 21
3.7 Amazon SageMaker 23
4 DESENVOLVIMENTO DA SOLUÇÃO 30
4.1 Projeto implementado 30
CONCLUSÃO 40
REFERÊNCIAS 41
GLOSSÁRIO 42

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 INTELIGÊNCIA ARTIFICIAL COMO SERVIÇO: EXPLORANDO OS RECURSOS COGNITIVOS DA AWS

O Processamento de Linguagem Natural (NLP — Natural Language Processing) é um campo de Inteligência Artificial que permite que máquinas compreendam, interpretem e gerem linguagem humana. Ele combina linguística computacional com Aprendizado de Máquina para processar grandes volumes de texto ou áudio (falas).

Com o uso de NLP é possível desenvolver soluções como:

- **Chatbots, Assistentes virtuais e Reconhecimento de voz.**
  - Exemplos: Alexa, Google Assistant e Siri.
- **Tradução automática.**
  - Exemplo: tradução de textos utilizando um smartphone.
- **Transcrição de textos.**
  - Exemplo: transcrição de áudio em conversas em um chat, como o WhatsApp.
- **Análise de sentimentos.**
  - Exemplo: monitoramento de redes sociais.
- **Pesquisa e extração de informações.**
  - Exemplo: motores de busca.
- **Geradores de conteúdo (Large Language Model — LLM).**
  - Exemplo: ChatGPT.

Neste capítulo, iremos abordar alguns conceitos fundamentais de NLP que são utilizados pelos serviços cognitivos da AWS como o Amazon Transcribe, Translate e Polly. Assim, conseguiremos propor uma solução multilíngue para um serviço de conversação na AWS.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.1 Projeto

A seguir, podemos visualizar o diagrama do projeto de uma solução que treina e utiliza um modelo de Machine Learning (ML) para classificar comentários de usuários sobre filmes.

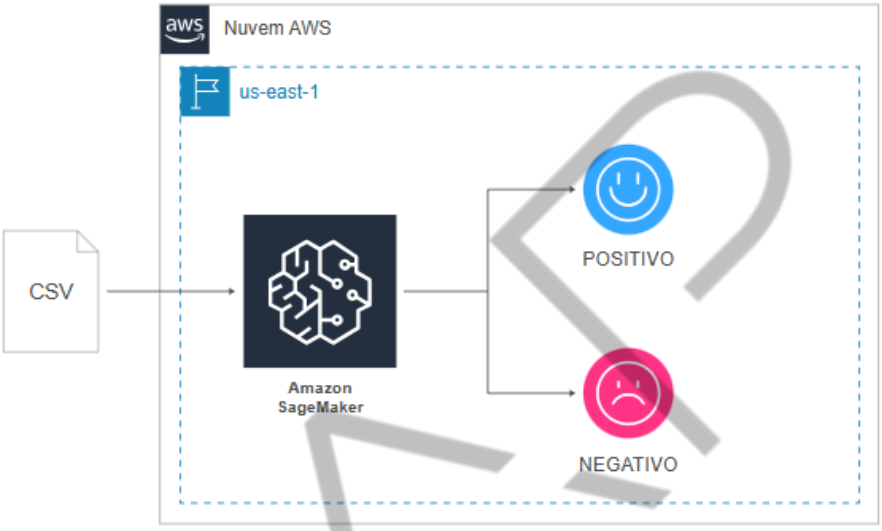

Figura 1 - Solução multilíngue
Fonte: Elaborado pelo autor (2025), adaptado por FIAP (2025)

Utilizaremos o **Amazon SageMaker** que permite criar experimentos de ML na AWS, além de conter um ambiente de desenvolvimento completo para desenvolver códigos em Python com Machine Learning, como o JupyterLab.

Com o **SageMaker**, iremos treinar um modelo de Machine Learning para classificar comentários entre positivos ou negativos de acordo com os textos dos comentários.

Apesar de criarmos nosso próprio modelo de Machine Learning, a AWS conta com diversos serviços de processamento de linguagem natural prontos para uso, como: o **Amazon Transcribe**, para realizar a transcrição de áudio; o **Amazon Translate** para realizar a tradução de um idioma em outro; e o **Amazon Polly** para sintetização de voz. Exploraremos também uma opção que utiliza IA Generativa como o **Amazon Bedrock**.

Primeiro iremos abordar os serviços disponíveis de NLP da AWS para que, em seguida, possamos criar nossa própria solução.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 PROCESSAMENTO DE LINGUAGEM NATURAL

O processamento de linguagem natural é um ramo da Inteligência Artificial que permite conferir às máquinas o entendimento de linguagens escritas e faladas.

Para isso, são implementadas técnicas de extração de significado a partir de textos, reconhecendo regras gramaticais, palavras e frases.

Além de melhorar a compreensão de textos, também permite entender como as palavras se relacionam em diferentes idiomas e auxiliar na desambiguação semântica.

2.1 Análise sintática

Observe o exemplo a seguir de uma árvore sintática:

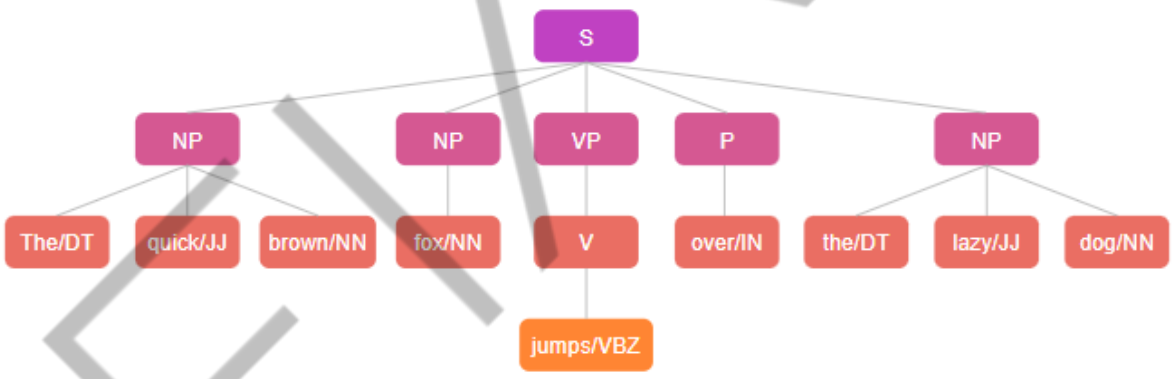

Figura 2 - Árvore Sintática
Fonte: AWS (2025), adaptado por FIAP (2025)

Ao quebrar uma frase, é possível rotular cada elemento em um processo conhecido como a **etiquetagem** (*tagging*), possibilitando a identificação de artigos, adjetivos, substantivos (exemplos: nomes, locais etc.), verbos, entre outros elementos de discurso.

Desse modo, é possível identificar ações a serem executadas, como verbos em uma frase, permitindo que dispositivos consigam ser acionados, como nos assistentes virtuais, por exemplo Alexa e Siri.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Um dos maiores desafios para os sistemas que usavam NLP era a falta de precisão e o entendimento do contexto. Porém, esse cenário vem mudando rapidamente com o surgimento de novas técnicas que conseguem entender o contexto com maior precisão.

Vejamos alguns pontos na evolução do processamento de linguagem natural na figura a seguir.

Figura 3 - Evolução de modelos de NLP
Fonte: AWS (2025), adaptado por FIAP (2025)

A figura anterior mostra parte da evolução das técnicas implementadas para o processamento de linguagem natural.

Desde a década de 1960, são aplicadas técnicas linguísticas para análise sintática e semântica.

Entre as décadas de 1960 e 1980 surgiram técnicas para melhorar a análise semântica das frases, como a Teoria de Dependência Conceitual (Conceptual Dependency Theory — CDT).

A partir dos anos 1990 até os anos de 2010, houve uma explosão de técnicas que aplicaram a Aprendizagem Profunda (Deep Learning) com diversas soluções como Redes Neurais Recorrentes (RNNs), Long Short-Term Memory (LSTM) e modelos como Sequence-to-Sequence (Seq2Seq) com aplicações como tradução e resumo de textos e a criação de agentes conversacionais (exemplo: chatbots).

Nas últimas décadas, com a grande disponibilização de dados e com o advento da era do Big Data, surgiram aplicações de NLP em larga escala e uso de computação sob demanda, com as novas arquiteturas que utilizam o grande poder computacional

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

alinhado com o paralelismo para processar simultaneamente grandes volumes de dados.

O uso de NLP ganhou relevância mundial com a chegada de modelos que usam Redes Neurais e Deep Learning, tendo como destaque a chegada dos Modelos de Linguagem Ampla (Large Language Models — LLMs), que teve em 2017, a publicação de um artigo científico intitulado *"Attention Is All You Need*", explicando sobre as camadas que consideram o contexto das frases digitadas.

A AWS oferece soluções que usam NLP para diversos cenários de uso. Vale mencionar que as soluções adotam Redes Neurais para melhoria de performance. A seguir iremos abordar alguns dos serviços oferecidos e seus conceitos.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 SOLUÇÕES DE NLP NA AWS

3.1 Amazon Transcribe

O **Amazon Transcribe** é um serviço do tipo Speech-To-Text (STT) que permite a transcrição de áudios e vídeos em textos. Ele utiliza Machine Learning para identificar as palavras e, então, gerar os conteúdos textuais.

Suas principais características são:

- **Reconhecimento de fala:** permite converter arquivos de áudio ou *streaming* em texto.
- **Suporte a múltiplos idiomas:** transcreve áudio em vários idiomas, incluindo português, inglês, espanhol e francês.
- **Suporte a pontuação e formatação:** pode adicionar vírgulas, pontos e formatação ao texto gerado para melhoria da legibilidade.
- **Identificação de diferentes narradores:** permite identificar múltiplas pessoas em um áudio.
- **Identificação de discursos tóxicos:** pode identificar insultos, assédio, discursos de ódio (somente nos Estados Unidos).
- **Transcrição médica especializada:** realiza a transcrição de falas de médicos e profissionais da saúde (somente nos Estados Unidos).

Os formatos suportados estão descritos no quadro a seguir.

	Processamento em Lote	***Streaming***
Formatos suportados:	AMR FLAC M4A MP3 MP4 Ogg WebM WAV	FLAC Ogg Opus PCM encoding
Formatos recomendados:	FLAC WAV com codificação PCM 16-bit	

Quadro 1 - Tipos de arquivos suportados no Amazon Transcribe
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O Amazon Transcribe suporta transcrição em tempo real, incluindo uma interface no próprio console da AWS.

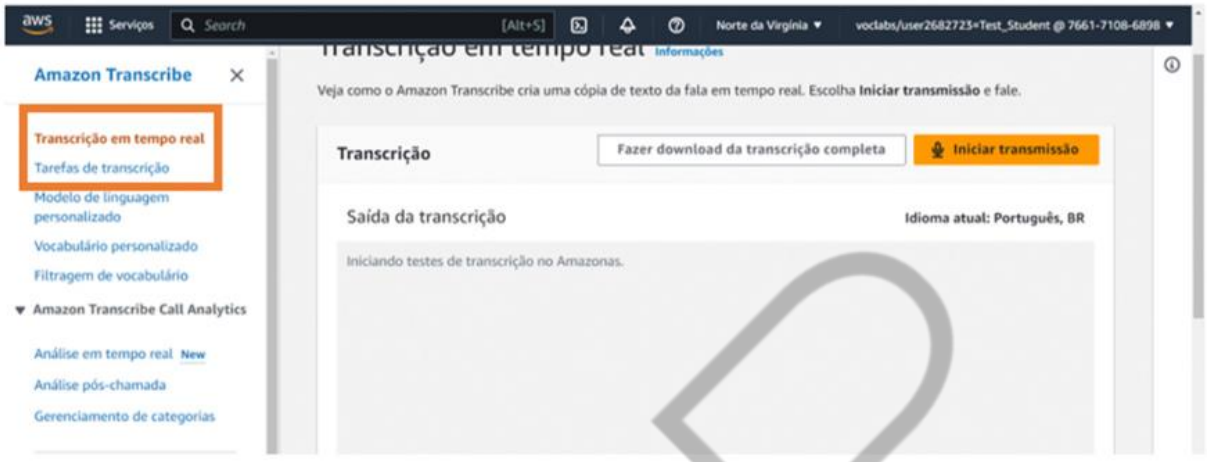

Figura 4 - Transcrição em tempo real
Fonte Elaborado pelo autor (2025)

Na figura “Transição em tempo real”, é possível observar o menu de acesso à funcionalidade, assim como um botão para iniciar a transcrição e captura de áudio. Além disso, é possível realizar o download da transcrição completa ou criar uma tarefa de transcrição.

Vejamos agora como criar uma tarefa de transcrição com o Transcribe.

3.2 Uso do Transcribe

A seguir, iremos demonstrar uma simulação de uma transcrição a partir de um áudio em inglês de um podcast da BBC.

Inicialmente, criamos um bucket S3 com o nome <suasiniciais>-entrada-audio e outro bucket com o nome <suasiniciais>-saida-audio.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

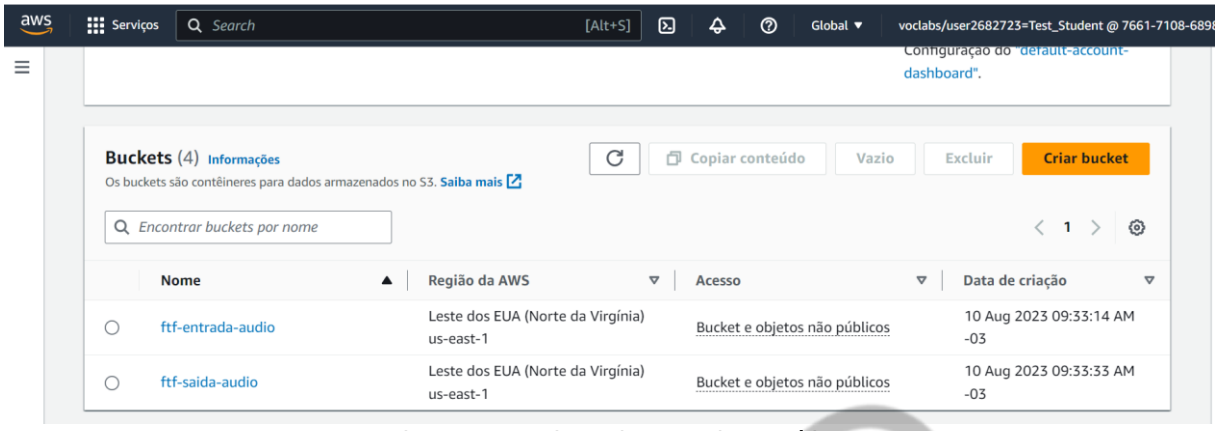

Figura 5 - Buckets de entrada e saída
Fonte: Elaborado pelo autor (2025)

Conforme observado na figura anterior, criamos dois buckets, sendo um para receber arquivos de áudio (ftf-entrada-audio) que serão transcritos e outro para receber a transcrição (ftf-saida-audio).

Em seguida, criaremos uma tarefa de transcrição. Para isso, precisamos acessar o **Amazon Transcribe** e clicar em “Criar tarefa”.

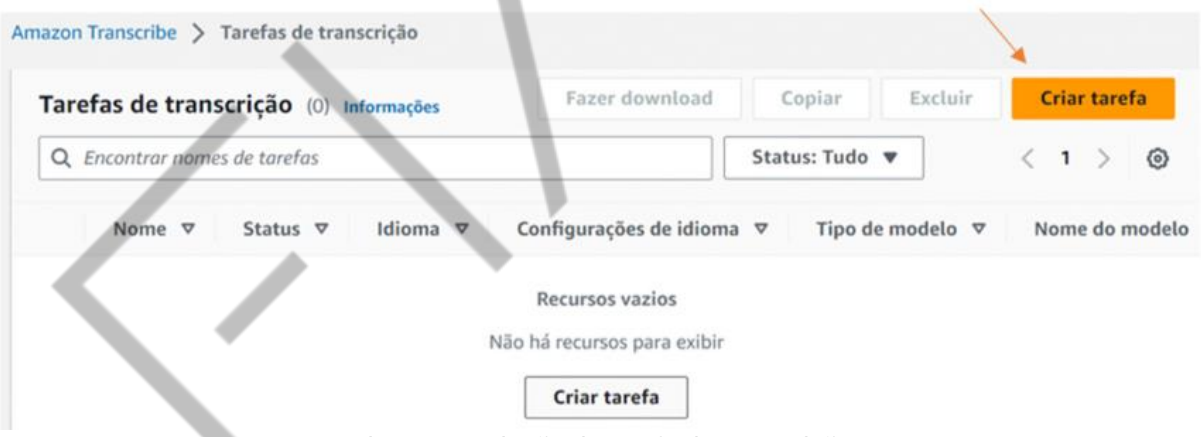

Figura 6 - Criação de tarefa de transcrição
Fonte: Elaborado pelo autor (2025)

Em seguida, definimos um nome para a tarefa de transcrição**.**

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

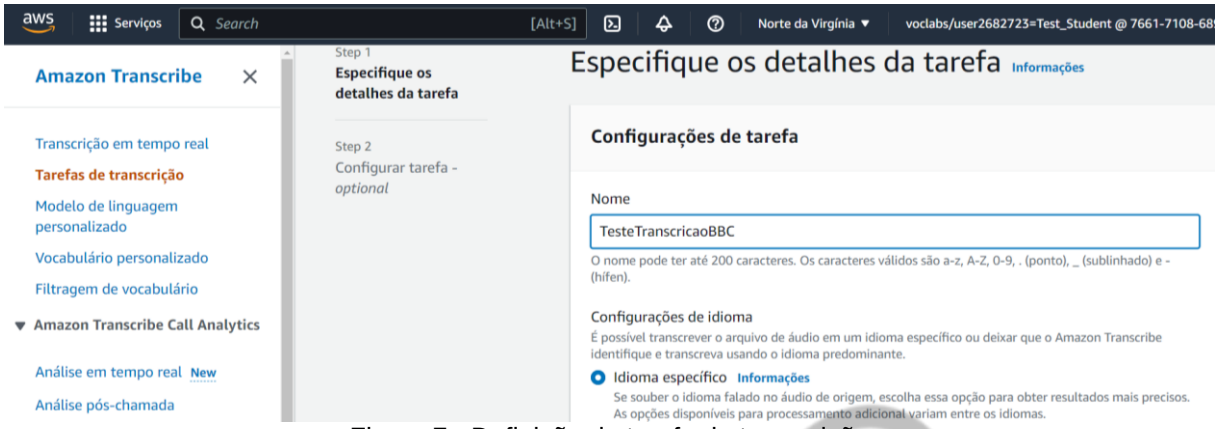

Figura 7 - Definição de tarefa de transcrição
Fonte: Elaborado pelo autor (2025)

Na figura anterior, definimos um nome para nossa tarefa como “TesteTranscricaoBBC”.

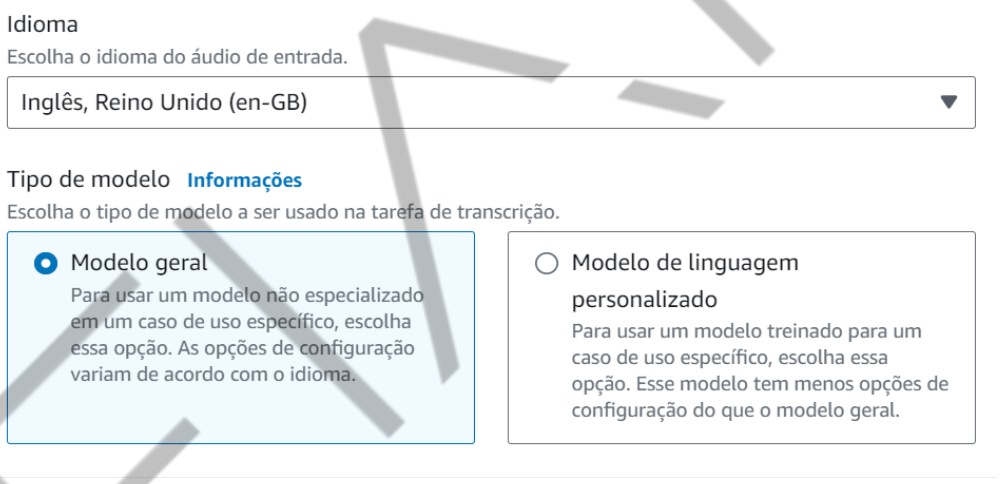

Figura 8 - Definição de idioma de entrada
Fonte: Elaborado pelo autor (2025)

É importante definirmos o idioma para “Inglês, Reino Unido (en-GB)” pois já sabemos o idioma de entrada e essa informação facilita a transcrição.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

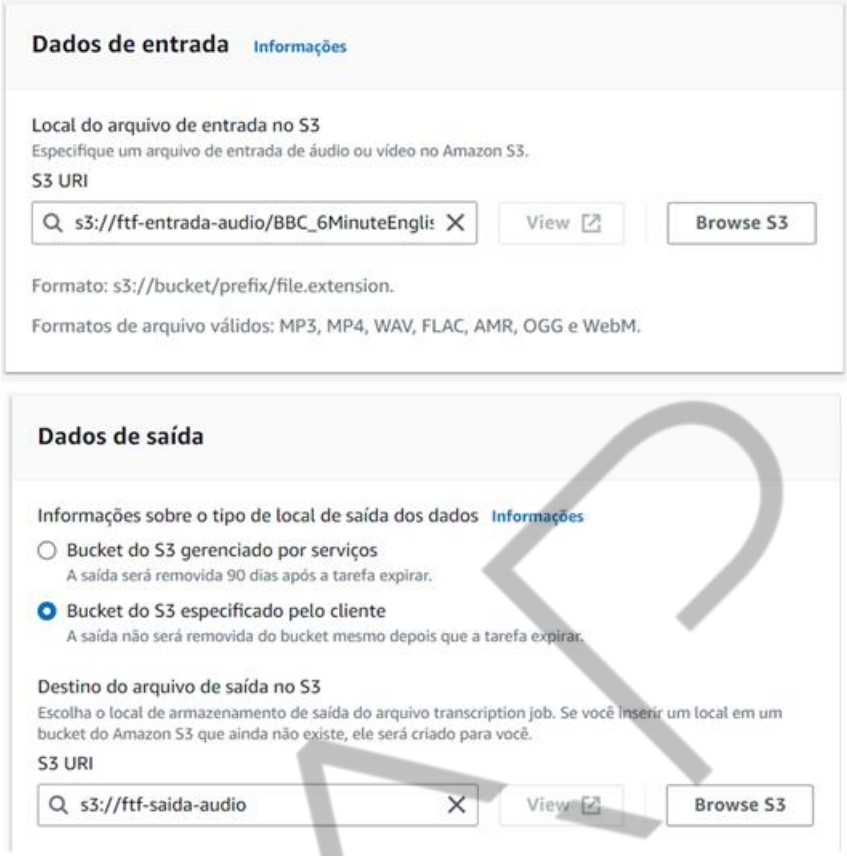

Figura 9 - Definição de bucket de entrada e saída
Fonte: Elaborado pelo autor (2025)

Como ilustrado na figura “Definição de bucket de entrada e saída”, precisamos escolher o caminho para o arquivo de entrada e o bucket para conter a saída da transcrição. Após isso, é necessário concluir a tarefa.

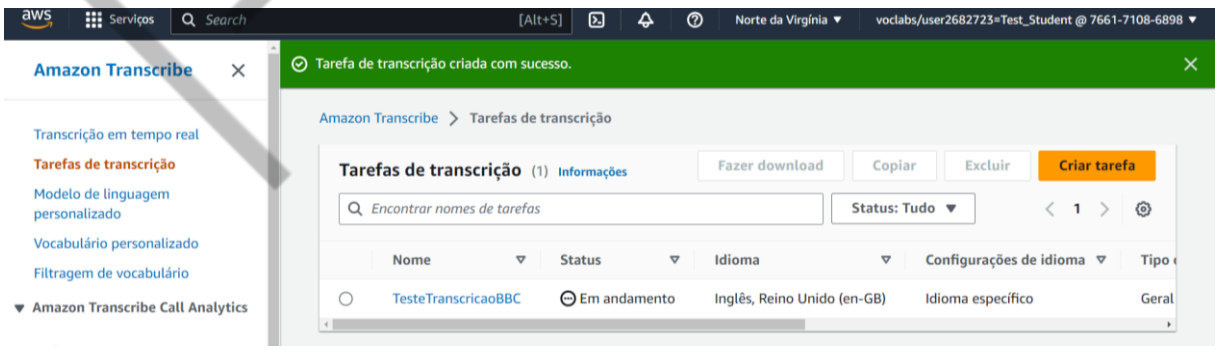

Figura 10 - Tarefa de transcrição
Fonte: Elaborado pelo autor (2025)

Após a conclusão da tarefa, acesse o bucket de saída.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

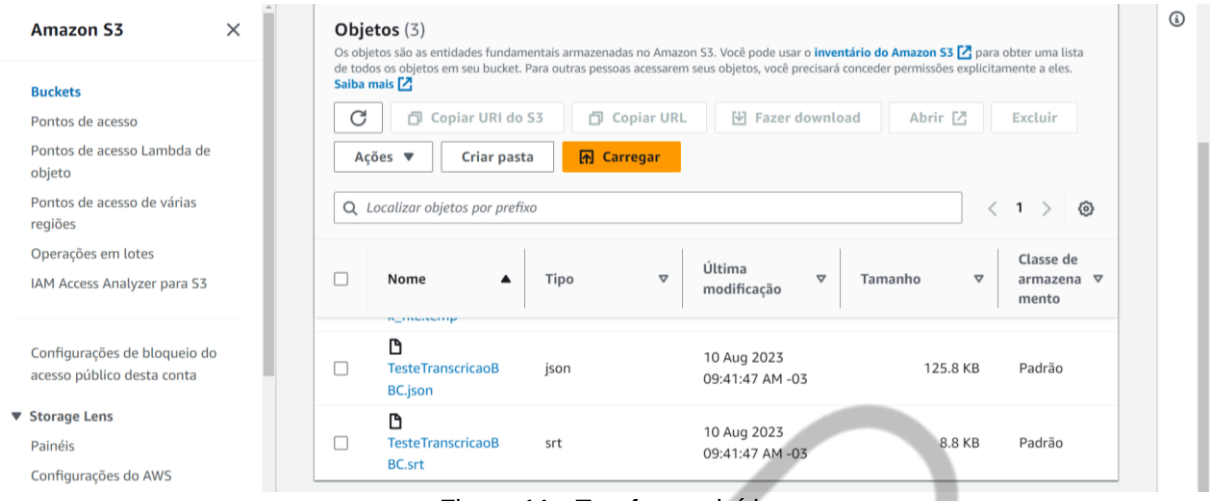

Figura 11 - Tarefa concluída
Fonte: Elaborado pelo autor (2025)

Na figura “Tarefa concluída”, podemos observar o resultado da transcrição. Temos dois arquivos gerados, sendo um no formato JSON e outro no formato SRT. O formato SRT é comumente utilizado para legendas, pois contém o horário de cada fala em um discurso.

A seguir, podemos observar o conteúdo do arquivo JSON gerado.


```
{
  "jobName": "TesteTranscricaoBBC",
  "accountId": "766171086898",
  "results": {
    "transcripts": [
      {
        "transcript": "This is the BBC. This podcast is
          supported by advertising outside the UK Smooth
          English from the BBC. Hello. This is six
          minutes English from BBC Learning English. I'm
          Beth and I'm Neil. Now, depending on how you
          look at it, Roy Sullivan was either the
          luckiest or the unluckiest man alive. Working
          as a US park ranger, Roy was struck by
          lightning on seven different occasions and
          survived them all. But Roy isn't the only
          victim of an unpredictable natural event
          sometimes called an act of God. In the last
          decade, an estimated half a million people have
```


Figura 12 - Conteúdo da transcrição
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Na figura “Conteúdo da transição”, podemos observar o conteúdo completo da transcrição do áudio. Na próxima figura, podemos ver o conteúdo do arquivo SRT.

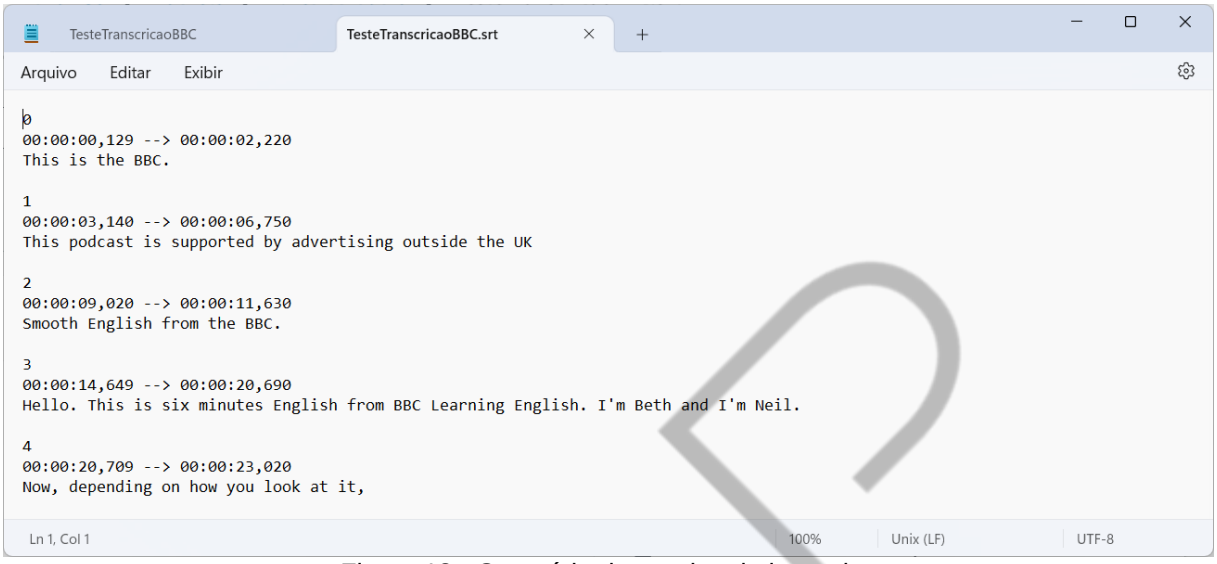

Figura 13 - Conteúdo do arquivo de legendas
Fonte: Elaborado pelo autor (2025)

Conforme demonstrado na figura “Conteúdo do arquivo de legendas”, temos o resultado da transcrição, sendo uma fala transcrita de cada vez de forma sequencial, incluindo hora, minuto e segundo a partir do início do áudio.

Assim, concluímos a demonstração do primeiro serviço de processamento de linguagem natural para transcrição de textos na AWS.

3.3 Amazon Translate

O **Amazon Translate** é um serviço da AWS baseado em Aprendizado de Máquina que realiza tradução em diversos idiomas. Ele permite traduzir texto em tempo real ou grandes volumes de texto em lote, sendo útil para aplicações multilíngues, suporte ao cliente, análise de conteúdo global, entre outros.

O Amazon Translate tem suporte a tradução com base em modelos estatísticos ou em modelos de redes neurais.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

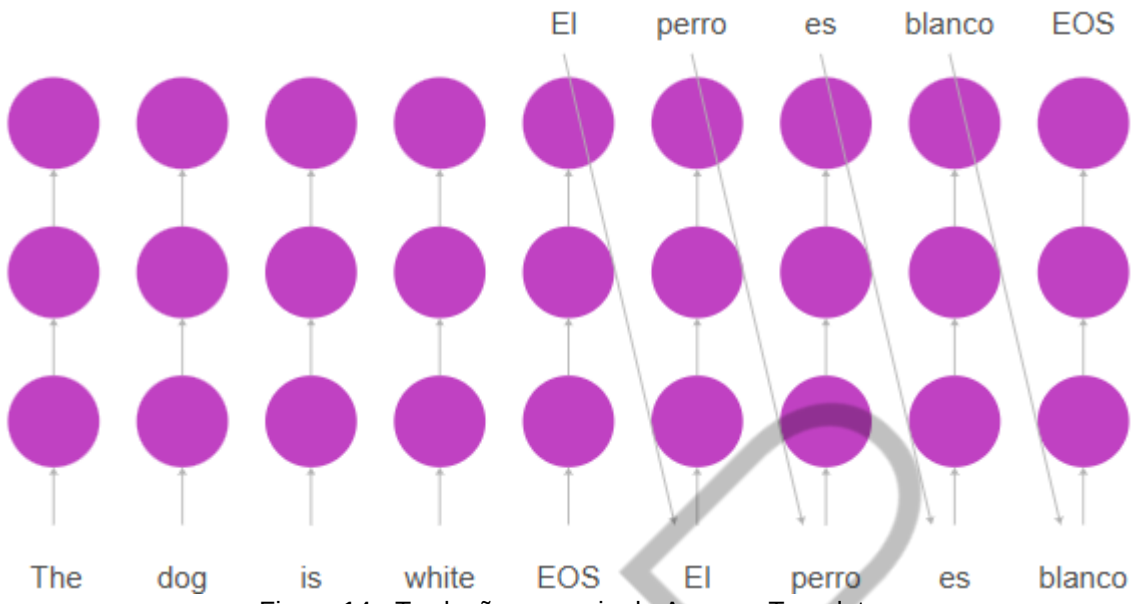

Figura 14 - Tradução por meio do Amazon Translate
Fonte: AWS (2025), adaptado por FIAP (2025)

Na figura anterior, podemos observar uma tradução por meio de uma rede neural, em que temos como dados de entrada, um texto em inglês e como saída, um texto em espanhol.

Além disso, ainda é possível realizar tradução em tempo real ou em lote. A seguir vemos um exemplo de tradução em tempo real no próprio console da AWS.

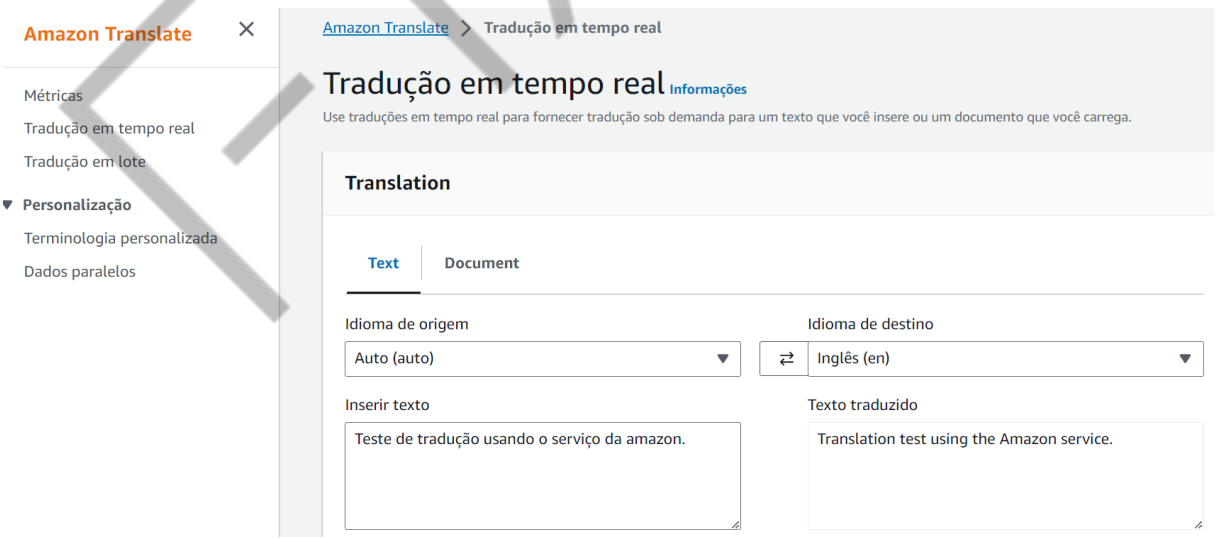

Figura 15 - Tradução em tempo real
Fonte: Elaborado pelo autor (2025)

Também é possível enviar um documento completo para tradução.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

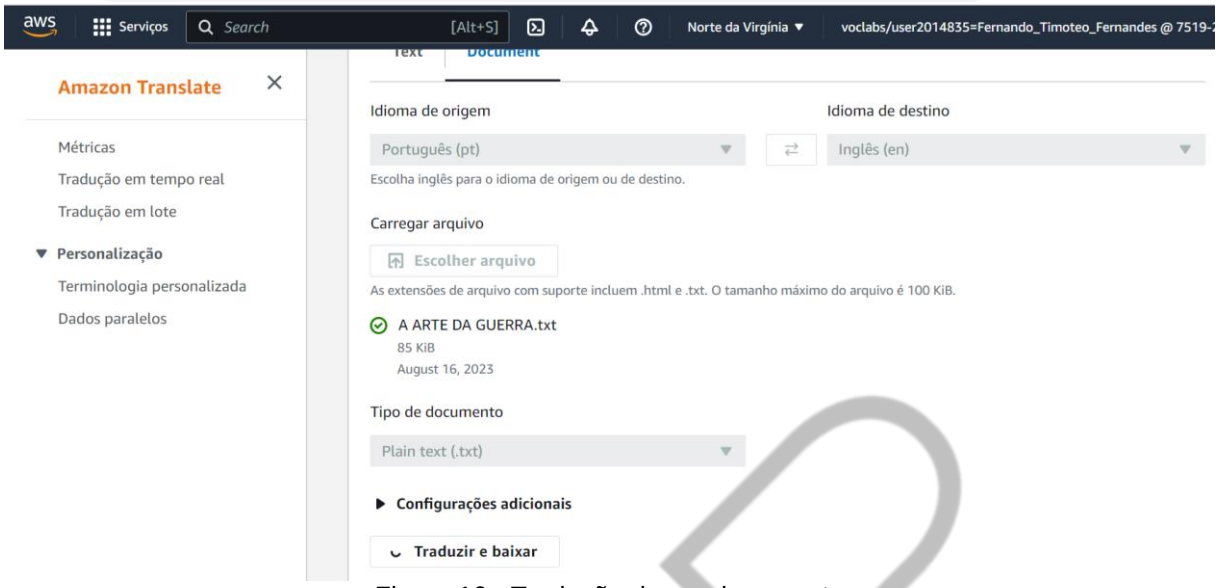

Figura 16 - Tradução de um documento
Fonte: Elaborado pelo autor (2025)

Portanto, com o Amazon Translate é possível realizar traduções de diversos idiomas, incluindo traduções mais precisas com o uso de redes neurais.

3.4 Amazon Comprehend

O **Amazon Comprehend** é um serviço de **processamento de linguagem natural** da AWS que utiliza o Aprendizado de Máquina para **analisar e extrair insights de texto**. Ele permite identificar sentimentos, tópicos, entidades, linguagem do texto, sem a necessidade de treinamento prévio em Machine Learning.

A seguir podemos observar como o Amazon Comprehend identifica termos e resume documentos de acordo com tópicos e categorias.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

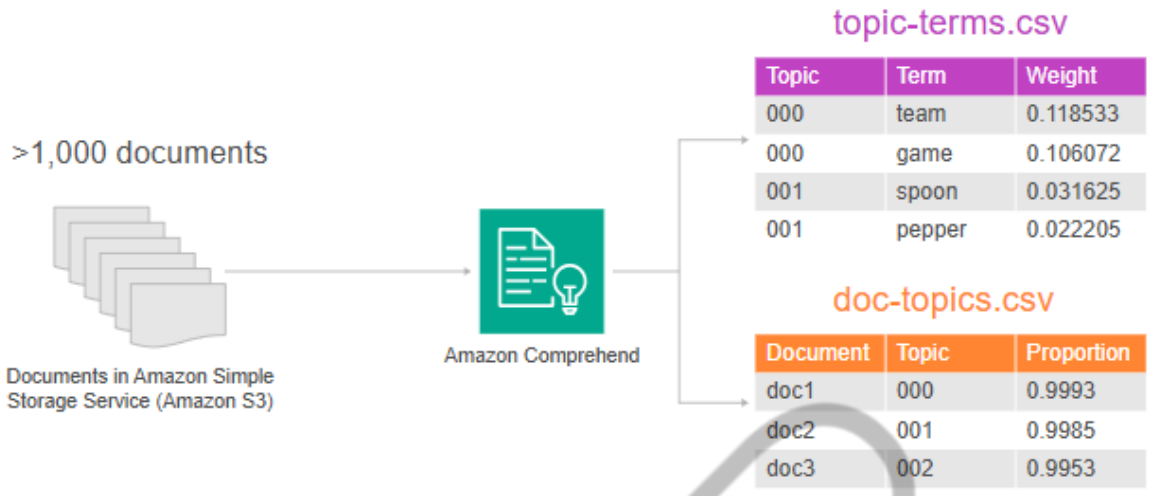

Figura 17 - Topic Modelling
Fonte: AWS (2025), adaptado por FIAP (2025)

Na figura “Topic Modelling”, é possível notar um conjunto de documentos enviados ao Amazon Comprehend, para identificar os principais termos em cada documento e o peso de cada termo.

Em um outro recurso do Amazon Comprehend, é possível analisar um texto e extrair as principais entidades, como nomes, locais e informações pessoais.

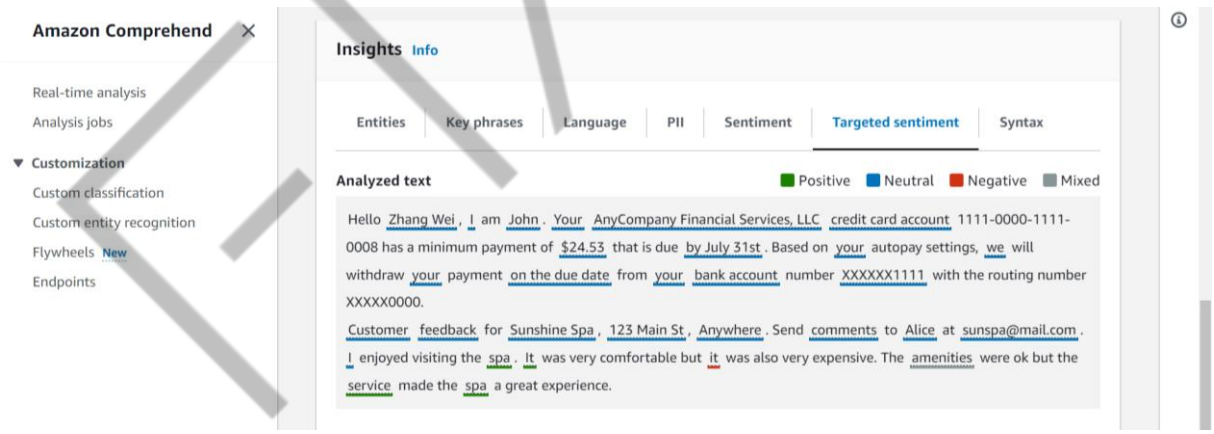

Figura 18 - Identificação de entidades
Fonte: Elaborado pelo autor (2025)

Na figura anterior, é possível observar que o serviço do Amazon Comprehend identifica pessoas, locais, telefones, valores e outras informações contidas no texto. Dessa forma, é possível classificar rapidamente documentos, além de identificar sentimentos do conteúdo do texto.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Alguns cenários de uso incluem a análise de sentimentos de redes sociais, com a identificação de discursos hostis. Outro cenário possível seria utilizar o Comprehend para analisar os comentários de clientes em relação aos produtos cadastrados em um e-commerce.

3.5 Amazon Polly

O **Amazon Polly** é um serviço da AWS que converte texto em fala (Text-To-Speech — TTS) usando Inteligência Artificial para gerar vozes naturais e realistas. Ele pode ser utilizado para criar aplicações interativas, como assistentes virtuais, audiobooks, além de permitir a acessibilidade para deficientes visuais.

O Amazon Polly contém recursos como:

- **Suporte a mais de 30 idiomas:** inclusive o português.
- **Personalização com SSML:** com o uso de tags SSML (Speech Synthesis Markup Language), é possível ajustar entonação e pausas ao texto.
- **Diferentes motores:** é possível usar geradores de fala neurais e até de IA Generativa.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

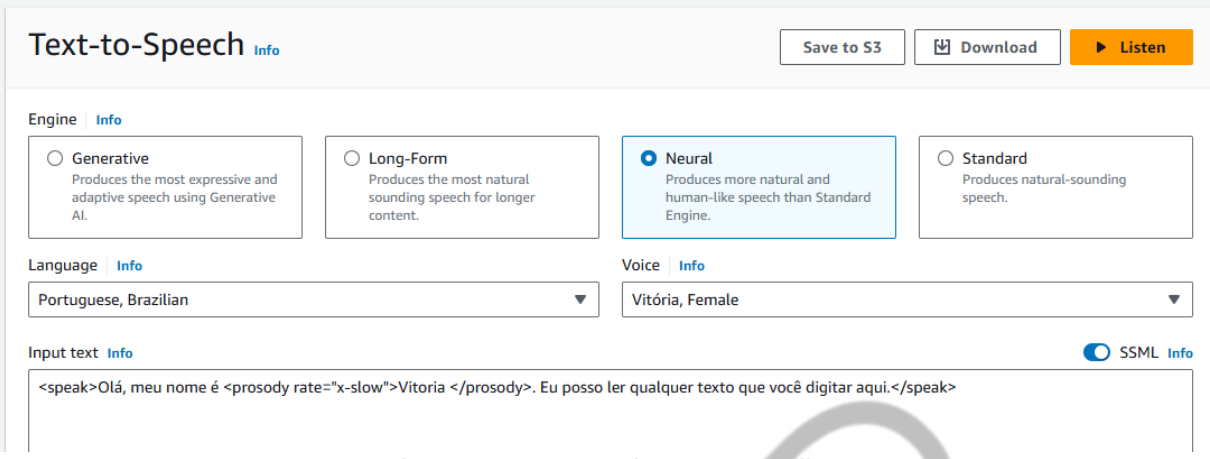

Figura 19 - Motores do Amazon Polly
Fonte: Elaborado pelo autor (2025)

Conforme ilustrado na figura “Motores do Amazon Polly”, é possível escolher entre diferentes motores de fala. Assim, você pode conferir as diferenças na geração de áudio.

Também podemos ver que está habilitada a marcação SSML, em que incluímos marcações (TAGs) no texto, como “<prosody>” para dar uma entonação diferente ao nome “Vitória”.

3.6 Amazon Bedrock

A Amazon disponibiliza o Amazon Bedrock que é um ambiente que oferece serviços de IA Generativa. Nele podemos construir diferentes modelos com bases em Modelos Fundacionais (*Foundation Models*) e de diversos provedores (exemplos: Llama da Meta, Claude da Anthropic, Titan da Amazon, entre outros).

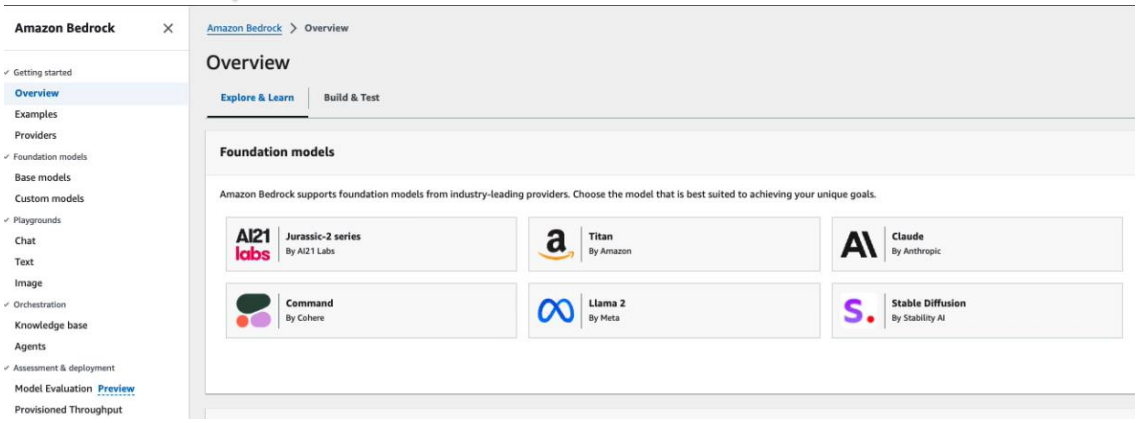

Figura 20 - Amazon Bedrock
Fonte: AWS (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Com o Amazon Bedrock, é possível desenvolver modelos para geração de texto, de imagens, chatbots e sumarização. Podemos escolher entre os seguintes playgrounds:

- **Playground de chat:** simular uma conversa com um chat.
- **Playground de imagem:** cria imagens com base nas instruções do *prompt*.
- **Playground de textos:** pode utilizar modelos de LLM pré-selecionados para executar tarefas de NLP.

Você pode selecionar os modelos que deseja utilizar, clicando na opção “Model access”.

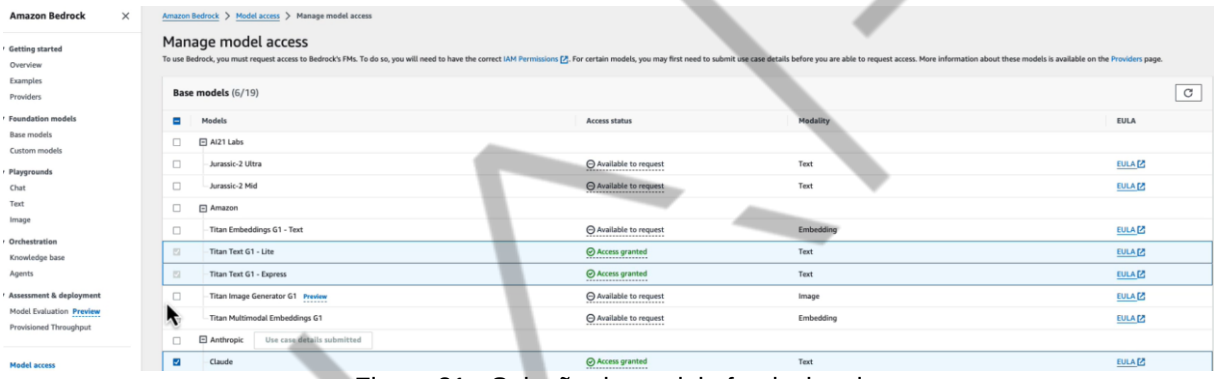

Figura 21 - Seleção de modelo fundacional
Fonte: AWS (2025)

Após a seleção, é possível testar os modelos usando os “playgrounds” disponíveis na interface do Bedrock.

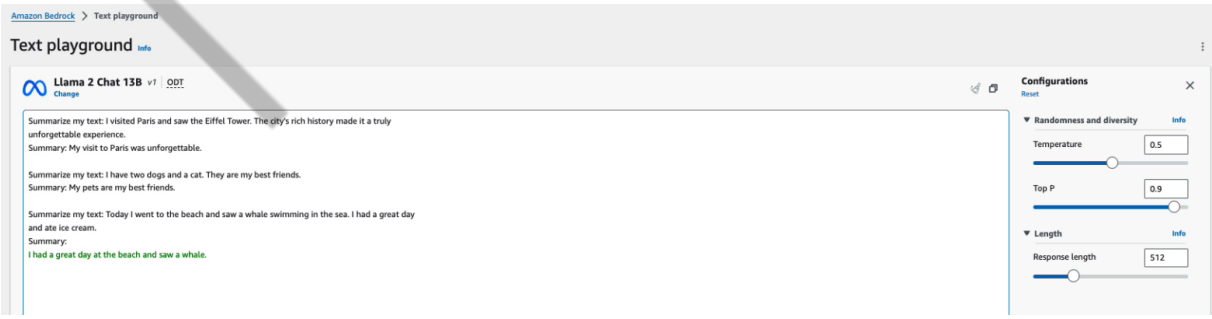

Figura 22 - Playground de Texto
Fonte: AWS (2025)

Na figura “Playground de Texto”, podemos observar alguns “prompts” de exemplo informados aos modelos. A partir disso, solicitamos que o modelo faça um

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

resumo de uma nova sentença (*summary*) e clicamos em executar. A resposta do modelo está em verde.

O modelo selecionado foi o LLama 2 da empresa Meta. No entanto, é possível mudar o modelo a qualquer momento, clicando em “Change”.

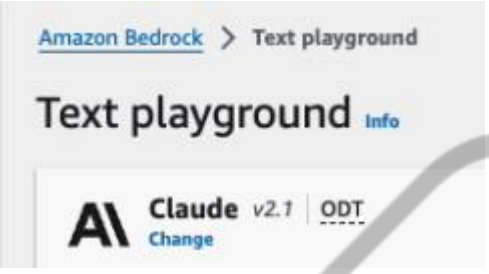

Figura 23 - Mudança de modelo de LLM
Fonte: Elaborado pelo autor (2025)

Na figura anterior, podemos visualizar a mudança de modelo selecionado para o modelo “Claude” da empresa Anthropic.

A seguir, vamos explorar o Amazon SageMaker para criarmos nossos próprios modelos de Machine Learning.

3.7 Amazon SageMaker

O **Amazon SageMaker** é um serviço da AWS que facilita o desenvolvimento, treinamento e implantação de modelos de Machine Learning e Inteligência Artificial de forma escalável e gerenciada. Ele reduz a complexidade da infraestrutura e permite que cientistas de dados e desenvolvedores foquem na construção de modelos eficientes.

O SageMaker possui diferentes recursos como:

- **AutoML:** possibilita o treinamento de modelos automaticamente a partir de dados, apenas selecionando o tipo de problema de Machine Learning e o desfecho pretendido.
- **MLOps:** possui integração para implantar modelos em tempo real e monitorar os recursos utilizados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **SageMaker JumpStart:** disponibiliza modelos pré-treinados e soluções prontas para acelerar o desenvolvimento

É possível também criar seu próprio notebook em Python no SageMaker, tanto de forma isolada quanto dentro de um ambiente de desenvolvimento como o JupyterLab.

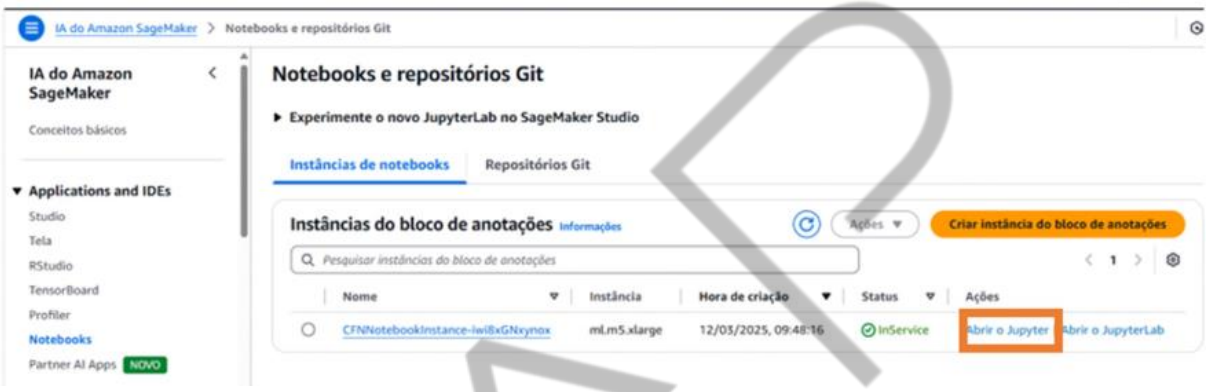

Figura 24 - Instâncias de Notebooks
Fonte: Elaborado pelo autor (2025)

Na figura “Instâncias de Notebooks”, podemos observar a tela do SageMaker AI, em que é possível visualizar “Instâncias do bloco de anotações” (notebooks em Python ou R).

Ao clicar em “Abrir o Jupyter”, você será direcionado para um ambiente de desenvolvimento, o “Jupyter”, que permite criar notebooks em Python ou R.

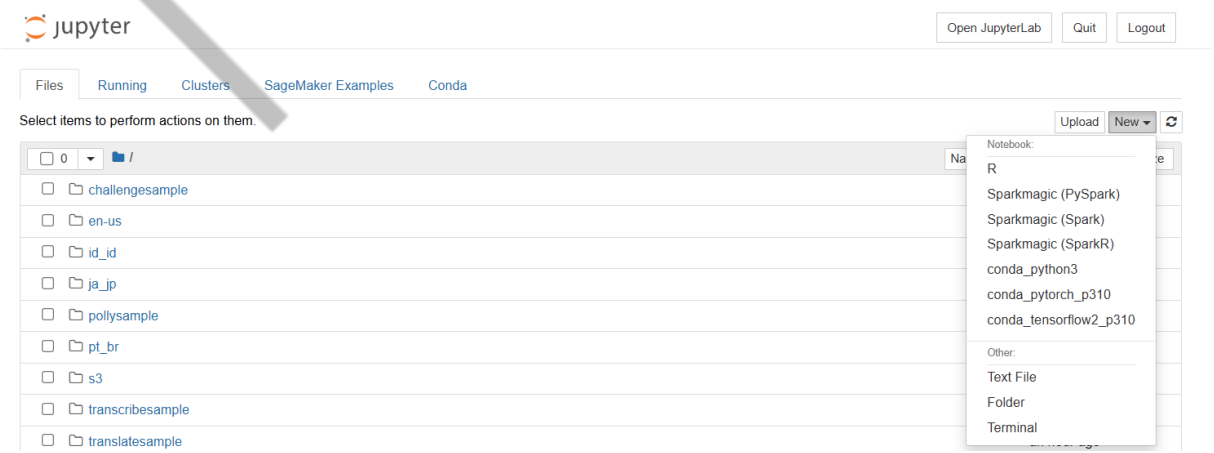

Figura 25 - Tela do Jupyter
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Selecione “conda_python3” e você será direcionado para um notebook em Python conforme exibido a seguir.

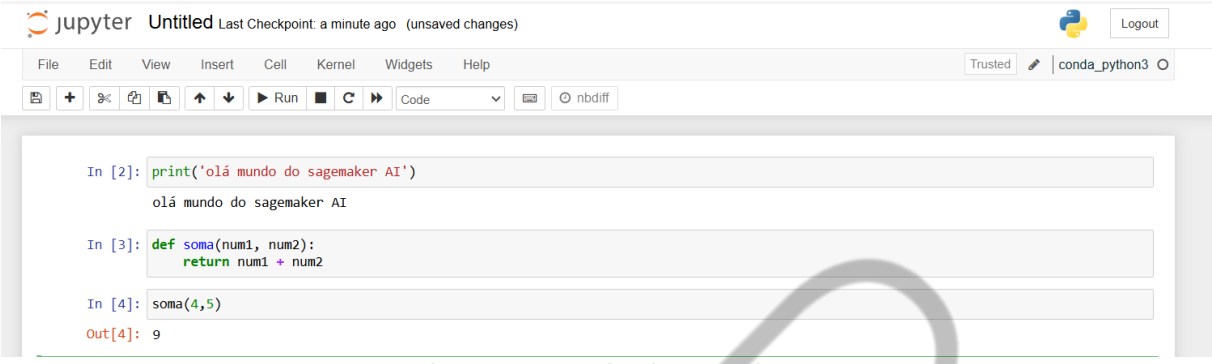

Figura 26 - Notebook em Python
Fonte: Elaborado pelo autor (2025)

Podemos observar na figura anterior alguns comandos executados em Python, diretamente no notebook criado dentro do Jupyter.

Este ambiente é ideal para o desenvolvimento colaborativo, como o treinamento de modelos de Machine Learning em consultorias, podendo compartilhar os notebooks com diversos colaboradores.

O código inteiro ficará seguramente armazenado na nuvem, evitando vazamento de dados.

Inicialmente, vamos criar uma instância de um bloco de anotações, escolhendo um nome, a família de instâncias disponível e a plataforma de execução.

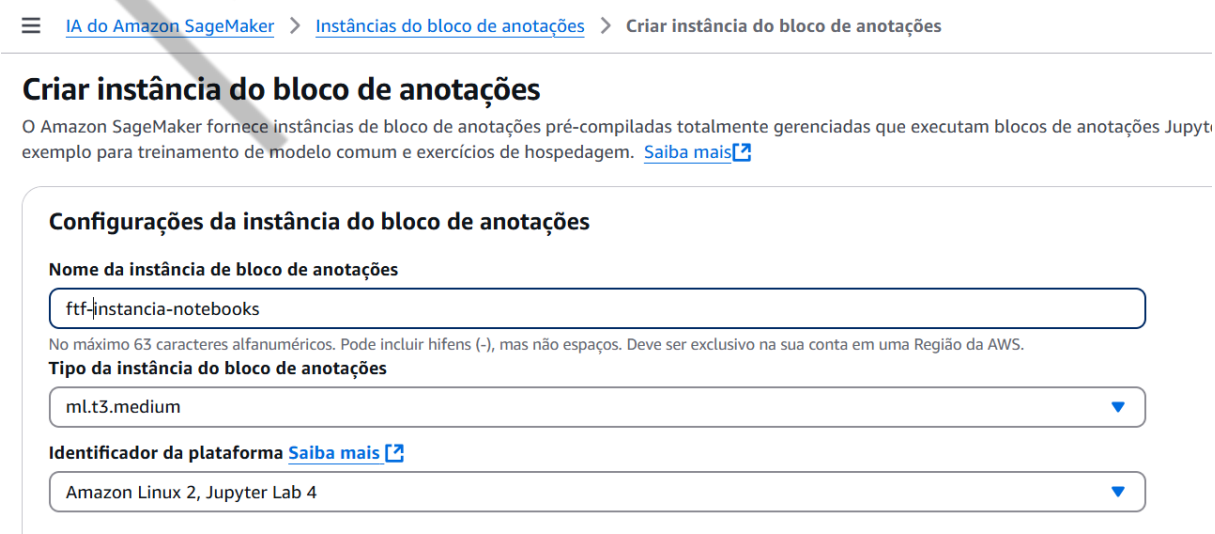

Figura 27 - Criação de bloco de anotações
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Na figura anterior, podemos observar a criação de uma instância de notebooks, o tipo de instância selecionado (ml.t3.medium) e a plataforma de execução (Amazon Linux 2, JupyterLab 4).

Após a criação da instância é possível criar notebooks em Python ou em R, usando ambientes como o JupyterLab. Para isso, basta selecionar a opção “Abrir o JupyterLab” ao lado da instância criada.

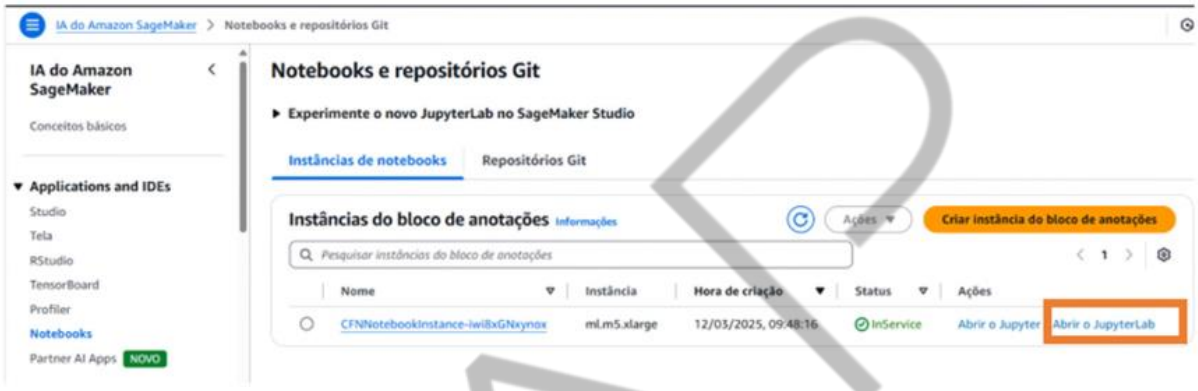

Figura 28 - Acesso ao JupyterLab
Fonte: Elaborado pelo autor (2025)

Nesta opção, você será direcionado para o ambiente de desenvolvimento do JupyterLab, conforme exibido a seguir.

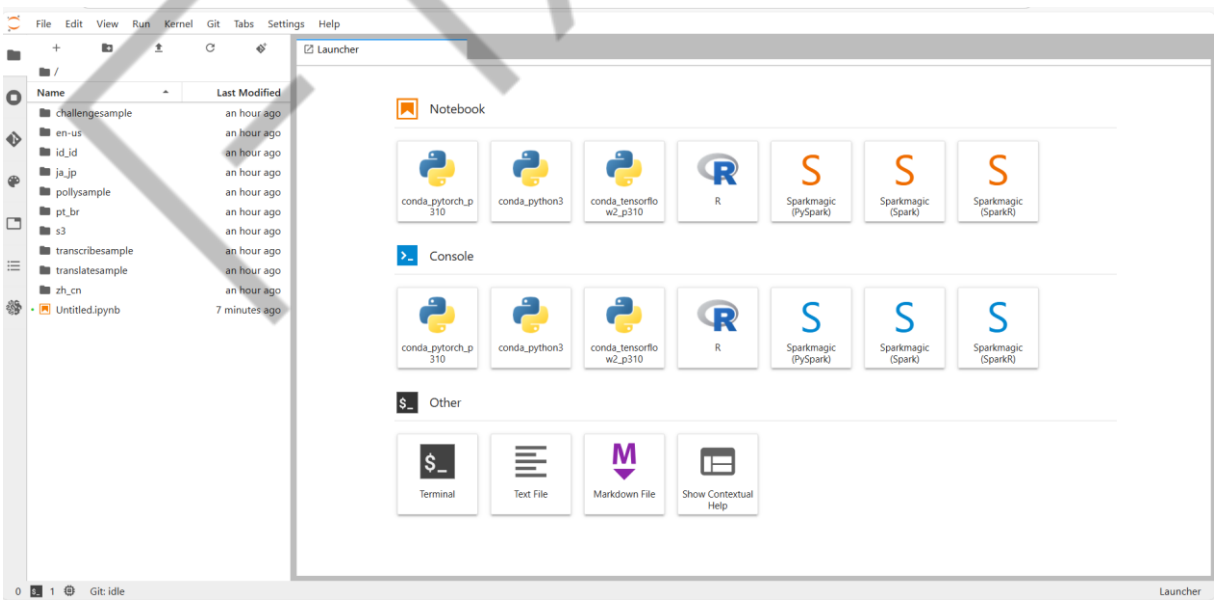

Figura 29 - Ambiente do JupyterLab
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Na figura anterior, podemos observar o ambiente do JupyterLab, em que é possível ter acessos a diversos recursos, como o Git e a exemplos e notebooks de exemplo disponibilizados pela AWS.

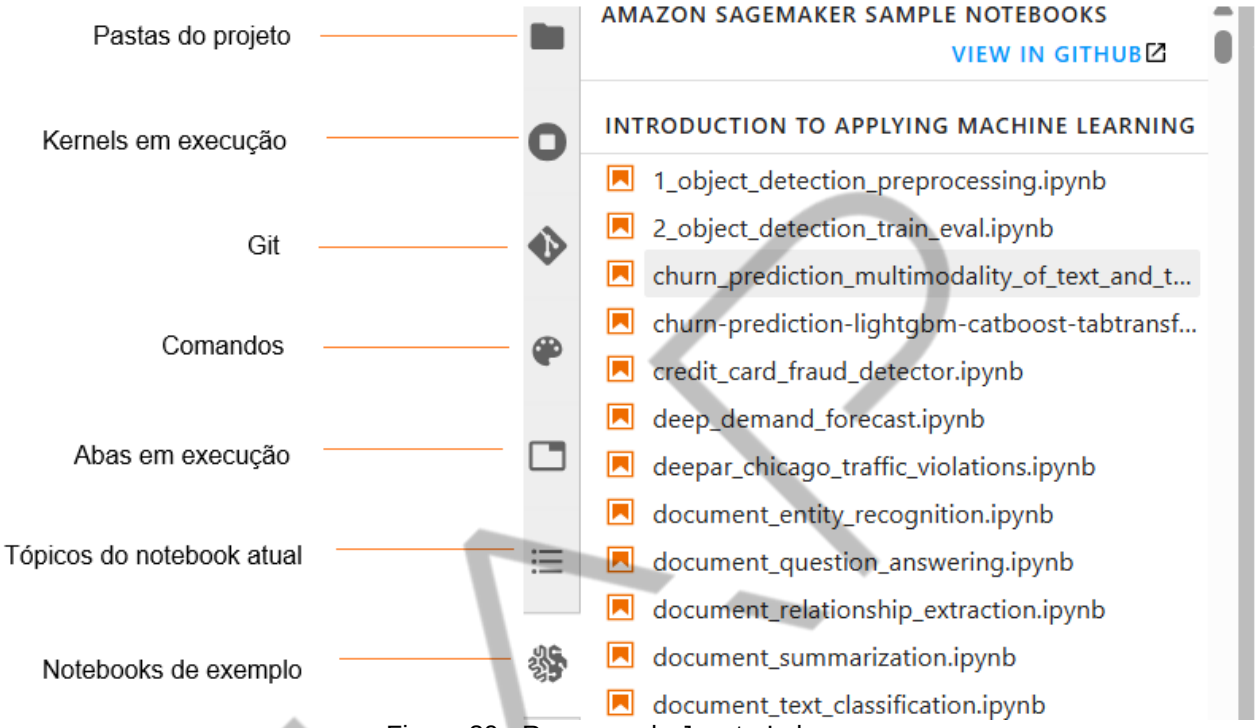

Figura 30 - Recursos do JupyterLab
Fonte: Elaborado pelo autor (2025)

Na figura “Recursos do JupyterLab”, podemos observar diversos recursos do JupyterLab, incluindo o acesso às pastas do projeto atual, visualização dos kernels dos notebooks em execução, acesso ao Git, acesso a comandos e ajuda, listagem de abas abertas, tópicos dos notebooks atuais e notebooks de exemplo.

Os notebooks criados isoladamente também aparecem no JupyterLab.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

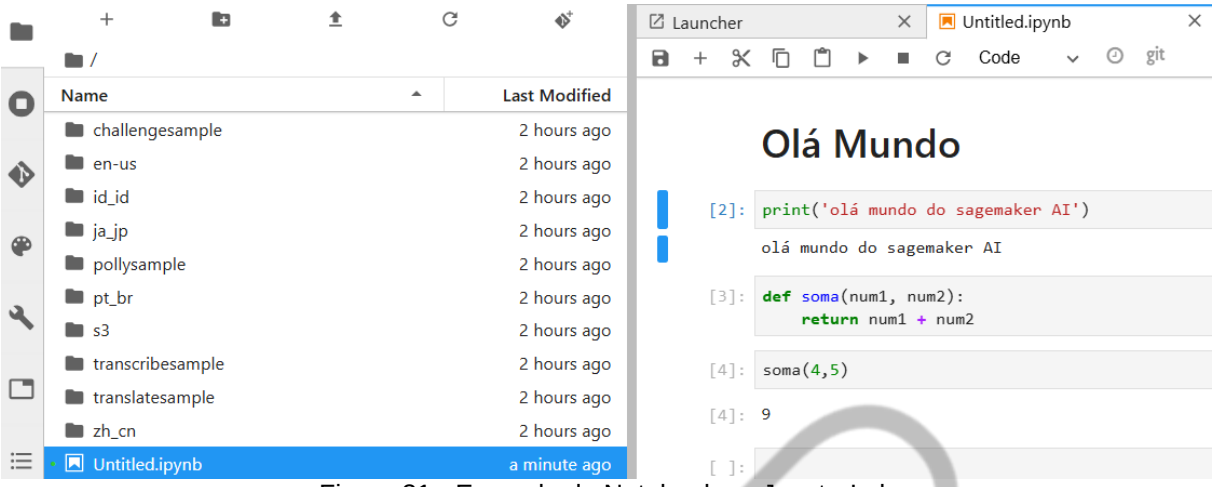

Figura 31 - Exemplo de Notebook no JupyterLab
Fonte: Elaborado pelo autor (2025)

Na figura “Exemplo de Notebook no JupyterLab”, podemos observar um notebook que ainda não foi nomeado (*Untitled*), criado no Jupyter com alguns códigos em Python em execução. Ao criar notebooks, eles aparecerão um ao lado do outro, como em abas de um navegador.

A seguir colocamos o código em Python para analisarmos com o Amazon Comprehend.

```
# Criando um DataFrame com uma coluna "comentario"
data = {
    "comentario": [
        "Gostei muito do filme Matrix, a história e os efeitos especiais são incríveis!",
        "Achei O Senhor dos Anéis muito longo e arrastado, não curti.",
        "Interestelar é um dos melhores filmes de ficção científica que já vi!"
    ]
}

df = pd.DataFrame(data)
pd.set_option('display.max_colwidth', None)
df.head()
```

Código-fonte 1 - Criação do DataFrame
Fonte: Elaborado pelo autor (2025)

Na figura a seguir, temos um exemplo do conjunto de dados criado e exibido por meio da biblioteca Pandas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	comentario
0	Gostei muito do filme Matrix, a história e os efeitos especiais são incríveis!
1	Achei O Senhor dos Anéis muito longo e arrastado, não curti.
2	Interestelar é um dos melhores filmes de ficção científica que já vi!

Figura 32 - DataFrame de exemplo
Fonte: Elaborado pelo autor (2025)

A seguir colocamos o código em Python em que usamos o Amazon Comprehend para analisar os sentimentos de cada sentença.

```
import boto3
import json

# Inicializando o cliente do Comprehend
comprehend = boto3.client(service_name='comprehend')

# Análise de sentimentos dos comentários
for n in range(len(df)):  # Percorrer todos os comentários
    text = df.iloc[n]['comentario']
    response  =  comprehend.detect_sentiment(Text=text,
LanguageCode='pt')
    sentiment = response['Sentiment']
    print(f'{sentiment} - {text}')
```

Código-fonte 2 - Uso do Amazon Comprehend no SageMaker
Fonte: Elaborado pelo autor (2025)

Veja que o Amazon Comprehend conseguiu detectar o principal sentimento de cada sentença.

```
POSITIVE - Gostei muito do filme Matrix, a história e os efeitos especiais são incríveis!
NEGATIVE - Achei O Senhor dos Anéis muito longo e arrastado, não curti.
POSITIVE - Interestelar é um dos melhores filmes de ficção científica que já vi!
```

Figura 33 - Sentimento identificado pelo Comprehend
Fonte: Elaborado pelo autor (2025)

Agora iremos criar nosso próprio modelo de NLP para identificar sentimentos em comentários de usuários.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 DESENVOLVIMENTO DA SOLUÇÃO

Neste momento iremos implementar uma solução de análise de sentimentos usando o Amazon SageMaker AI.

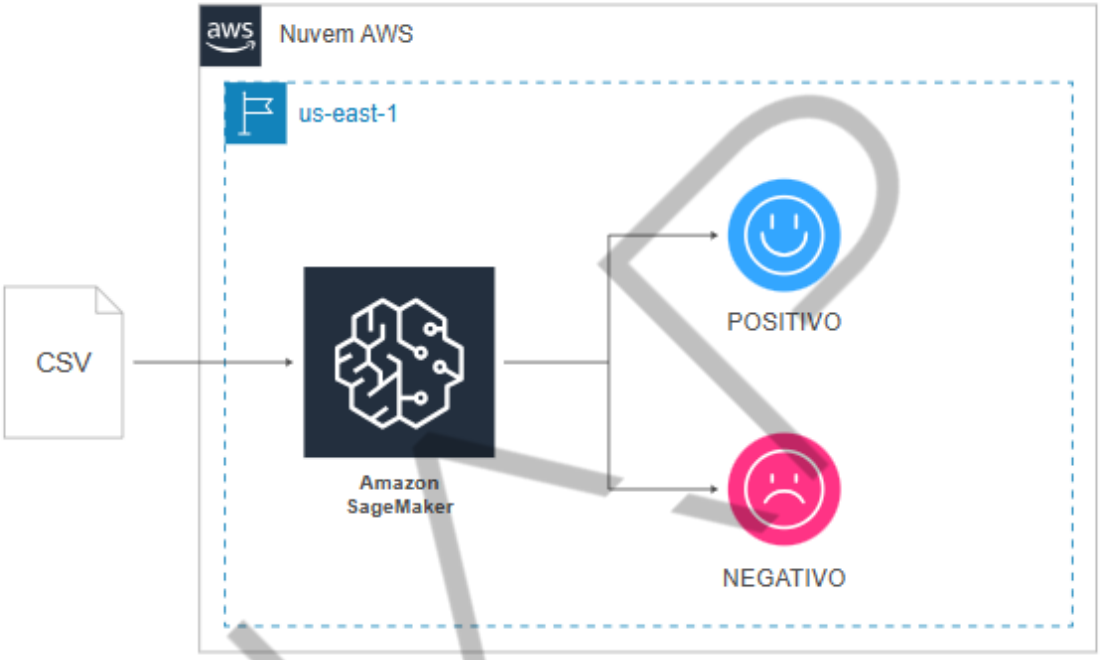

Figura 34 - Solução de NLP para classificar sentimentos
Fonte: Elaborado pelo autor (2025), adaptado por FIAP (2025)

Vamos utilizar como base de dados, os comentários de usuários do site IMDB. O conjunto de dados (dataset) está disponível publicamente no site Kaggle: <https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews/data>.

4.1 Projeto implementado

Acesse o SageMaker AI, e clique no menu “Notebooks”. Em seguida, acesse o JupyterLab e crie um notebook em Python (conda_python).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

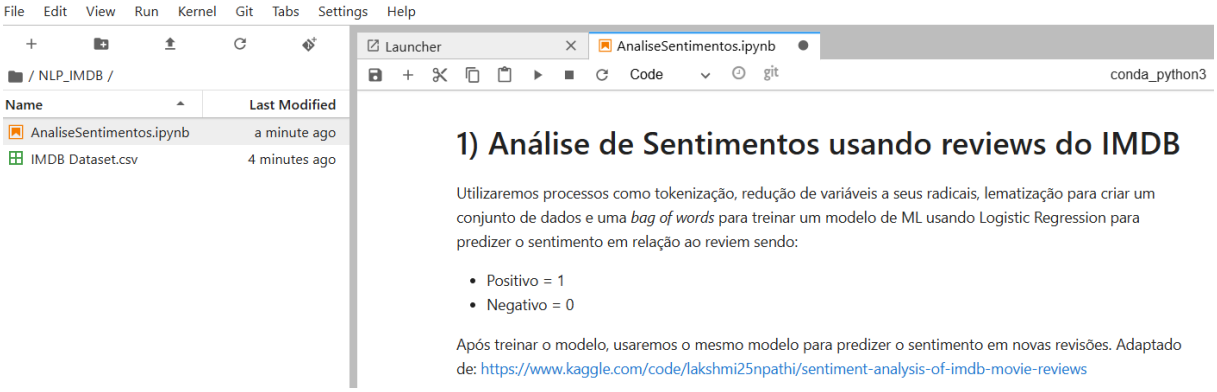

Figura 35 - Notebook para análise de sentimentos criado
Fonte: Elaborado pelo autor (2025)

Na figura anterior, vimos um notebook criado chamado “AnaliseSentimentos.ipynb”. Também realizamos o upload de um arquivo CSV para treinarmos um modelo de NLP.

Em seguida, adicione o código a seguir para importar as bibliotecas necessárias:

```
%%time
import pandas as pd
import numpy as np
import nltk
import re
from bs4 import BeautifulSoup
from nltk.tokenize.toktok import ToktokTokenizer
from nltk.corpus import stopwords
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.preprocessing import LabelBinarizer
from sklearn.metrics import accuracy_score
```

Código-fonte 1 - Importação de bibliotecas
Fonte: Elaborado pelo autor (2025)

No código anterior, importamos as bibliotecas necessárias para realizar o processamento de linguagem natural, como o “nltk”, o “re” e o “bs4” para realizar a limpeza de dados. Também realizamos a importação de bibliotecas do Scikit-Learn, que é a biblioteca do Python que permite treinar modelos de Machine Learning como a Regressão Logística (*Logistic Regression*) e criar processos reprodutíveis com o uso de **pipelines**, separando etapas do processamento.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vamos iniciar a exploração dos dados usando a biblioteca Pandas.

```
     #importar palavras de parada
nltk.download('stopwords')

     #carga dos dados
     imdb_data=pd.read_csv('IMDB Dataset.csv')

#visualização dos dados
     print(imdb_data.shape)
     imdb_data.head(10)
```

Código-fonte 2 - Visualização dos dados com Pandas
Fonte: Elaborado pelo autor (2025)

No código anterior, importamos as “palavras de parada” (*stop words*) para remoção do texto. As *stop words* são palavras comuns que não carregam significado, sendo irrelevantes para treinar nosso modelo de ML. Utilizaremos estas palavras para ações de limpeza de dados, evitando contabilizar palavras irrelevantes.

Após a importação das *stop words*, exibimos parte dos dados, com o uso da biblioteca Pandas que permite visualizar o formato dos dados e o conjunto de dados em formato tabular, conforme a figura a seguir.

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive
5	Probably my all-time favorite movie, a story o...	positive
6	I sure would like to see a resurrection of a u...	positive
7	This show was an amazing, fresh & innovative i...	negative
8	Encouraged by the positive comments about this...	negative
9	If you like original gut wrenching laughter yo...	positive

Figura 36 - Conteúdo do dataset
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Também podemos observar a distribuição dos dados por meio do seguinte código:

```
     #Vejo a frequência de distribuição dos dados
     imdb_data['sentiment'].value_counts()

     #saída                    do                    resultado
#positive 5000
     #negative 5000
```

Código-fonte 3 - Frequência dos sentimentos nos dados em análise
Fonte: Elaborado pelo autor (2025)

A seguir iremos dividir uma parte dos dados para treino e outra parte para testes.

```
     # Divisão do conjunto de dados
     # Dados de Treino (80%)
     train_reviews=imdb_data.review[:40000]
     train_sentiments=imdb_data.sentiment[:40000]

     # Dados de Testes (20%)
     test_reviews=imdb_data.review[40000:]
     test_sentiments=imdb_data.sentiment[40000:]
```

Código-fonte 4 - Limpeza de ruídos
Fonte: Elaborado pelo autor (2025)

No código anterior, separamos parte dos dados para treino (80%) e o restante para testes (20%).

```
     # Transformar os labels em 0 (negativo) e 1 (positivo)
     lb = LabelBinarizer()
     train_sentiments_bin                                    =
lb.fit_transform(train_sentiments)
     test_sentiments_bin = lb.transform(test_sentiments)

     # Lista de stopwords
     stopword_list = stopwords.words('english')
     tokenizer = ToktokTokenizer()

     # Transformer personalizado para pré-processamento
     class TextCleaner(BaseEstimator, TransformerMixin):
         def __init__(self):
             self.ps = nltk.PorterStemmer()

         def remover_html(self, text):
             soup = BeautifulSoup(text, "html.parser")
             return soup.get_text()
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        def remover_conteudo_colchetes(self, text):
            return re.sub('\[[^]]*\]', '', text)

        def remover_caracteres_especial(self, text):
            pattern = r'[^a-zA-Z0-9\s]'
            return re.sub(pattern, '', text)

        def remover_stopwords(self, text):
            tokens = tokenizer.tokenize(text)
            tokens = [token.strip() for token in tokens]
             filtered_tokens = [token for token in tokens if
token.lower() not in stopword_list]
            return ' '.join(filtered_tokens)

        def stemming(self, text):
             return ' '.join([self.ps.stem(word) for word in
text.split()])

        def clean_text(self, text):
            text = self.remover_html(text)
            text = self.remover_conteudo_colchetes(text)
            text = self.remover_caracteres_especial(text)
            text = self.remover_stopwords(text)
            text = self.stemming(text)
            return text

        def fit(self, X, y=None):
            return self

        def transform(self, X, y=None):
            return X.apply(self.clean_text)
```

Código-fonte 5 - Classe para limpeza de dados
Fonte: Elaborado pelo autor (2025)

No código anterior utilizamos a classe **LabelBinarizer** para dicotomizar o desfecho, sendo o valor 0 para sentimentos negativos e 1 para sentimentos positivos.

Em seguida, criamos uma classe para realizar a remoção de ruídos nos dados, como caracteres especiais, pontos de parada e conteúdo em html.

Após a limpeza de ruídos, a classe irá transformar as palavras em seus radicais, em um processo conhecido como *stemming.* A seguir, vamos ver o que acontecerá com o dataset após o processo de limpeza e *stemming.*

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
imdb_data['review'].head()
```

```
0    one review mention watch 1 oz episod youll hoo...
1    wonder littl product film techniqu unassum old...
2    thought wonder way spend time hot summer weeke...
3    basic there famili littl boy jake think there ...
4    petter mattei love time money visual stun film...
Name: review, dtype: object
```

Figura 37 - Dataset após limpeza
Fonte: Elaborado pelo autor (2025)

Podemos observar uma mudança no dataset anterior, pois não há mais palavras de ligação (exemplo: *of*, *the*, *a*, entre outros), pois foram removidos durante o processo de limpeza de dados e do processo de *stemming*.

Agora iremos preparar um pipeline, onde automatizaremos todo o processo de limpeza de dados.

O pipeline irá definir etapas como limpeza de dados, vetorização e treinamento do modelo. No processo de limpeza de dados, será chamada a classe “TextCleaner” ao qual criamos para remover os ruídos nos dados e realizar o *stemming*.

Para a vetorização, utilizaremos a classe “CountVectorizer” que transforma cada sentença em um conjunto de palavras soltas, com a contagem de suas frequências, criando o que chamamos de Bag of Words (BoW).

Por último, usamos a classe LogisticRegression para treinar o nosso modelo preditivo.

```
%%time
#Criando o pipeline completo
pipeline = Pipeline([
    ('limpeza', TextCleaner()),
          ('vetorizacao',    CountVectorizer(min_df=0.0,
max_df=1.0, binary=False, ngram_range=(1,3))),
      ('classificador',  LogisticRegression(max_iter=500,
random_state=42))
])
```

Código-fonte 6 - Limpeza de ruídos
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O código anterior cria o pipeline de transformação e classificação dos dados para que seja utilizado para treinamento do modelo e, em seguida, para realizar novas predições. Agora, precisamos aplicar o pipeline em nossos dados.

```
%%time
# Treinamento do pipeline
pipeline.fit(train_reviews, train_sentiments_bin)

# Avaliação do pipeline
predictions = pipeline.predict(test_reviews)
acc = accuracy_score(test_sentiments_bin, predictions)
print("Acurácia do modelo:", acc)
# Acurácia do modelo: 0.8964
```

Código-fonte 7 - Treinamento do modelo
Fonte: Elaborado pelo autor (2025)

O código anterior aplica todo o pipeline de transformação e treina o algoritmo de regressão logística. Após isso, o modelo é treinado para realizar novas predições. Ao término do treinamento, medimos a acurácia do modelo. A acurácia ficou em torno de 0.89 ou 90%, mostrando boa performance preditiva do modelo. O processo pode levar em torno de 5 a 10 minutos.

Vejamos como utilizar o modelo treinado para realizar predições em novos dados ainda não vistos.

```
# --- Nova review positiva para teste ---
new_review = ["The Lord of the Rings was awesome, I´ve
never seen such an amazing movie!"]

# Previsão do sentimento da nova review
pred_new = pipeline.predict(pd.Series(new_review))

#Transforma a predição de 0 em negativo e 1 em positivo
pred_label = lb.inverse_transform(pred_new)
print("Sentimento da nova review:", pred_label[0])
# Sentimento da nova review: positive
```

Código-fonte 8 - Predição em novos dados
Fonte: Elaborado pelo autor (2025)

O processo aplica o pipeline de transformação no novo comentário e em seguida usa o modelo treinado para predizer o sentimento do comentário.

Observe o código completo:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
%%time
import pandas as pd
import numpy as np
import nltk
import re
from bs4 import BeautifulSoup
from nltk.tokenize.toktok import ToktokTokenizer
from nltk.corpus import stopwords
from sklearn.pipeline import Pipeline
from        sklearn.feature_extraction.text        import
CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.preprocessing import LabelBinarizer
from sklearn.metrics import accuracy_score
#importação das stopwords
nltk.download('stopwords')
# Leitura dos dados
imdb_data = pd.read_csv('IMDB Dataset.csv')
print(imdb_data.shape)
imdb_data.head(10)
# Divisão do conjunto de dados
train_reviews = imdb_data.review[:40000]
train_sentiments = imdb_data.sentiment[:40000]

test_reviews = imdb_data.review[40000:]
test_sentiments = imdb_data.sentiment[40000:]

# Transformar os labels em 0 (negativo) e 1 (positivo)
lb = LabelBinarizer()
train_sentiments_bin                                    =
lb.fit_transform(train_sentiments)
test_sentiments_bin = lb.transform(test_sentiments)

# Lista de stopwords
stopword_list = stopwords.words('english')
tokenizer = ToktokTokenizer()

# Transformer personalizado para pré-processamento
class TextCleaner(BaseEstimator, TransformerMixin):
    def __init__(self):
        self.ps = nltk.PorterStemmer()

    def remover_html(self, text):
        soup = BeautifulSoup(text, "html.parser")
        return soup.get_text()

    def remover_conteudo_colchetes(self, text):
        return re.sub('\[[^]]*\]', '', text)

    def remover_caracteres_especial(self, text):
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            pattern = r'[^a-zA-Z0-9\s]'
            return re.sub(pattern, '', text)

        def remover_stopwords(self, text):
            tokens = tokenizer.tokenize(text)
            tokens = [token.strip() for token in tokens]
            filtered_tokens = [token for token in tokens if
token.lower() not in stopword_list]
            return ' '.join(filtered_tokens)

        def stemming(self, text):
            return ' '.join([self.ps.stem(word) for word in
text.split()])

        def clean_text(self, text):
            text = self.remover_html(text)
            text = self.remover_conteudo_colchetes(text)
            text = self.remover_caracteres_especial(text)
            text = self.remover_stopwords(text)
            text = self.stemming(text)
            return text

        def fit(self, X, y=None):
            return self

        def transform(self, X, y=None):
            return X.apply(self.clean_text)

    # Criando o pipeline completo
    pipeline = Pipeline([
        ('limpeza', TextCleaner()),
        ('vetorizacao',         CountVectorizer(min_df=0.0,
max_df=1.0, binary=False, ngram_range=(1,3))),
        ('classificador',   LogisticRegression(max_iter=500,
random_state=42))
    ])

    # Treinamento do pipeline
    pipeline.fit(train_reviews, train_sentiments_bin)

    # Avaliação do pipeline
    predictions = pipeline.predict(test_reviews)
    acc = accuracy_score(test_sentiments_bin, predictions)
    print("Acurácia do modelo:", acc)
    # --- Nova review positiva para teste ---
    new_review = ["The Lord of the Rings was awesome, I´ve
never seen such an amazing movie!"]

    # Previsão do sentimento da nova review
    pred_new = pipeline.predict(pd.Series(new_review))
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
#Transforma a predição de 0 em negativo e 1 em positivo
pred_label = lb.inverse_transform(pred_new)
print("Sentimento da nova review:", pred_label[0])
# Sentimento da nova review: positive
```

Código-fonte 9 - Código completo da solução de NLP
Fonte: Elaborado pelo autor (2025)

Pronto! Usamos o SageMaker IA para criar um modelo completo de NLP para classificação de sentimentos em comentários de usuários sobre filmes.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CONCLUSÃO

Iniciamos mostrando as diversas soluções cognitivas da AWS para processamento de linguagem natural.

Vimos como o Transcribe pode ser usado para transcrever textos, o Translate para traduzir textos e o Polly para sintetização de textos.

Além disso, aprendemos soluções que extraem informações dos textos como o Amazon Comprehend e o Amazon Bedrock que utilizam IA Generativa para diversos cenários, como geração de textos, análise de textos e geração de imagens.

Por último, criamos uma solução completa de NLP para análise de comentários de filmes, mostrando como é fácil trabalhar com IA na AWS de forma segura e rápida.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

AWS Academy. **AWS Academy**, 2025. Disponível em: <https://awsacademy.instructure.com/>. Acesso em: 20 mar. 2025.

CULKIN, J.; ZAZON, M. **AWS Cookbook. Recipes for Success on AWS**. O' Reilly, 2022.

EAGAR, G. **Data Engineering with AWS**. Birmingham, UK: Packt Publishing, 2021.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7º Ed. São Paulo: Pearson Education do Brasil, 2018.

KAGGLE. **Sentiment analysis IMDB**. Disponível em: <https://www.kaggle.com/search?q=sentiment+analysis+IMDB>. Acesso em: 16 mar. 2025.

VASWANI, A. *et al*. **Attention Is All You Need**. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS (NeurIPS 2017), 2017, Long Beach, CA. Proceedings [...], 2017. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>. Acesso em: 20 mar. 2025.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

AWS	Amazon Web Services.
CLI	Command Line Interface.
EBS	Elastic Block Storage.
EC2	Elastic Compute Cloud.
EFS	Elastic File System.
RDS	Relational Database Service.
ELT	Extração, Carga e Transformação.
ETL	Extração, Transformação e Carga.
S3	Amazon Simple Storage Service.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com